StreamCast — Backend ↔ Frontend Wiring

How the Angular UI talks to the Contract, Clause, Partner, and Distribution microservices, end to end

> **UI:** `streamcast-ui` (Angular 17, standalone, signals) · `http://localhost:4200`
> **Edge:** API Gateway · `http://localhost:8082` (Spring Cloud Gateway + global JWT filter)
> **Discovery:** Eureka · `http://localhost:8761`
> **Services covered:** Contract-Service (8088), Clause-Service (8089), Partner-Service (8087), Distribution-Delivery (8083)
> **JWT secret:** shared across gateway + every service (HS256, 1 hour expiry)

1 · The shared transport — how every request travels

Every request below follows the same physical path. The four services only differ in *what* they store and *which roles* they accept. Understanding this once means the per-service sections can focus on shape and intent.

```
[Browser :4200]
   |
   |  Angular Service (e.g. ContractsService) builds:
   |    method + URL = environment.apiBase + "/contracts"
   |    headers = { Content-Type: application/json }
   |
   |  HttpClient request is intercepted by `authInterceptor`
   |    -> attaches  Authorization: Bearer <JWT>  from auth.user()?.token
   |
   v
[API Gateway :8082]   Spring Cloud Gateway
   1. CorsWebFilter    : allows http://localhost:4200, methods, headers
   2. JwtAuthFilter    : validates JWT (HS256, shared secret), extracts
                         email + role, injects them as headers:
                            X-User-Email: ...
                            X-User-Role : ROLE_ADMIN
                         Public paths skipped: /auth/login, /auth/register,
                         /auth/verify, /auth/forgot-password,
                         /auth/reset-password, /auth/forgot-username
   3. Predicate match  : /contracts/** -> lb://Contract-Service
                         /clauses/**   -> lb://Clause-Service
                         /partners/**  -> lb://Partner-Service
                         /manifests/** -> lb://Distribution-Delivery
   4. Load-balance     : asks Eureka for an instance, forwards request
   |
   v
[Microservice]
   - Spring Security @PreAuthorize checks the role
     (e.g. hasAnyRole('ADMIN','RIGHTS_MANAGER'))
   - Controller -> Service -> Repository -> MySQL
   - Returns either:
        APIResponse<T>  { message, data, success }   (Contract / Clause / Partner)
        OR raw entity                                  (Distribution)
   |
   v
[Angular Service] unwraps APIResponse via `.pipe(map(r => r.data))`
                  emits typed objects to the component
   |
   v
[Component] updates a signal -> template re-renders
```

1.1 · Role mapping — one source of truth, three enforcement points

A role string travels from the JWT claim all the way down to the controller. Each layer checks the same value:

Layer	Where	Action
1. UI — route guard	`app.routes.ts` → `roleGuard([...])`	Stops navigation; sends to `/forbidden`.
2. UI — sidebar & chip	`shell.component.ts` → `visibleSections()`	Hides links the user can't use. Cosmetic only.
3. Gateway — JWT filter	`JwtAuthFilter.java`	Rejects un-signed / expired tokens with 401 before reaching the service.
4. Service — `@PreAuthorize`	each controller method	Returns 403 if the role isn't in the allowed set.

> **Why four layers?** The two UI layers are convenience — they keep a user from seeing a 403 banner. The two backend layers are the security boundary; a bypassed UI cannot bypass them.

2 · Contract Service

Contracts represent licensing deals attached to a catalog title — territory, dates, exclusivity, and a text summary. The full CRUD set is exposed and the response is always wrapped in an `APIResponse` envelope.

2.1 · Service facts

Eureka name	`Contract-Service`
Port (direct)	`8088`
Database	`contract_database` (MySQL)
Base URL via gateway	`http://localhost:8082/contracts`
Response envelope	`APIResponse<T> = { message, data, success }`

2.2 · Endpoints — with role gates

Verb	Path	Roles allowed	Description
POST	`/contracts`	ADMIN, RIGHTS_MANAGER	Create a contract.
GET	`/contracts`	ADMIN, RIGHTS_MANAGER, SCHEDULER, COMPLIANCE_OFFICER	List all contracts.
GET	`/contracts/{id}`	same as above	Fetch one contract.
PUT	`/contracts/{id}`	ADMIN, RIGHTS_MANAGER	Replace fields on a contract.
DELETE	`/contracts/{id}`	ADMIN	Hard delete.

2.3 · Field-by-field mapping — UI form ↔ Java entity

UI form control	UI interface (`Contract`)	Backend DTO (`ContractRequestDTO`)	Entity column	Validation
Title ID (number)	`titleId: number`	`String titleId`	`long titleId`	`@Positive` (backend) · `min(1)` (UI)
Territories (text)	`territoryListJson: string`	`String territoryListJson`	`String`	`@NotBlank`
Start date	`startDate: string`	`String startDate`	`LocalDate` `yyyy-MM-dd`	`@NotNull`
End date	`endDate: string`	`String endDate`	`LocalDate`	must be *after* startDate (`@AssertTrue`)
Exclusive deal (checkbox)	`exclusivityFlag: boolean`	`Boolean exclusivityFlag`	`Boolean`	`@NotNull`
Terms summary	`termsSummary: string`	`String termsSummary`	`String`	`@Size(5,200)`
Status (dropdown)	`status: 'ACTIVE' | 'INACTIVE'`	`String status`	`String`	`@Pattern(ACTIVE|INACTIVE)`

2.4 · Frontend wiring — the two files that matter

CONTRACTS.SERVICE.TS

```
private base = `${environment.apiBase}/contracts`;     // http://localhost:8082/contracts

list()  → GET    /contracts           .pipe(map(r => r.data))
get(id) → GET    /contracts/{id}      .pipe(map(r => r.data))
create()→ POST   /contracts           .pipe(map(r => r.data))
update()→ PUT    /contracts/{id}      .pipe(map(r => r.data))
delete()→ DELETE /contracts/{id}
```

The `.pipe(map(r => r.data))` step is what unwraps the `APIResponse` envelope into plain typed objects, so components never see the `message` / `success` wrapper.

CONTRACTS.COMPONENT.TS

- **ContractsComponent** — signal-based list page. On `refresh()` it calls `contractsService.list()` and stores the result in `rows = signal<Contract[]>([])`.
- **ContractFormDialog** — a Material dialog with a reactive form whose validators *mirror* the backend rules (min(1) for titleId, minLength(5)/maxLength(200) for termsSummary, etc.). On save it returns the form value; the page subscribes to `create()` or `update()`, then re-runs `refresh()`.

2.5 · End-to-end — creating a contract

```
1. User clicks "New contract" in /contracts
2. Dialog opens (ContractFormDialog), user fills the form
3. On Save, the page calls:
       contractsService.create(formValue).subscribe(...)
4. HttpClient builds the request, authInterceptor adds:
       Authorization: Bearer eyJ...
5. Gateway: JwtAuthFilter decodes claims, sets X-User-Role: ROLE_RIGHTS_MANAGER
6. Gateway forwards to lb://Contract-Service via Eureka
7. Spring Security checks @PreAuthorize("hasAnyRole('ADMIN','RIGHTS_MANAGER')")
8. ContractController.createContract():
       - maps DTO -> entity via ContractRequestMapper
       - contractService.saveContract(contract)
       - wraps result in APIResponse("Contract created successfully", dto, true)
9. UI service unwraps .data, the page snackbars "Contract created" and refreshes
```

3 · Clause Service

A clause is a typed legal rule (geo-restriction, royalty split, holdback, etc.) tied to a parent contract. Unlike the other three services this controller uses verb-suffixed paths (`/clauses/post` , `/clauses/get`) rather than RESTful conventions.

3.1 · Service facts

Eureka name	`Clause-Service`
Port (direct)	`8089`
Database	`clause_database` (MySQL)
Base URL via gateway	`http://localhost:8082/clauses`
Cross-service dep.	`ContractClient` — a Feign client pointing to `Contract-Service` (currently a placeholder for future contract-existence checks).

3.2 · Endpoints

Verb	Path	Roles allowed	Description
POST	`/clauses/post`	ADMIN RIGHTS_MANAGER	Create a clause.
GET	`/clauses/get`	ADMIN RIGHTS_MANAGER SCHEDULER COMPLIANCE_OFFICER	List all clauses.

Path quirk: the controller uses `/post` and `/get` as suffixes rather than HTTP verbs alone. The Angular service mirrors that exactly — `list()` calls `/clauses/get` and `create()` calls `/clauses/post` . Don't "fix" this on one side without the other.

3.3 · Field-by-field mapping

UI interface (`Clause`)	Backend DTO (`ClauseRequestDTO`)	Notes
`clauseId?: number`	(returned only)	Server-assigned on create.
`clauseType: string`	`String clauseType`	e.g. `GEO_RESTRICTION` , `HOLDBACK` .
`detailsJSON: string`	`String detailsJSON`	Free-form JSON payload describing the rule.
`effectiveFrom: string`	`String effectiveFrom`	ISO date.
`effectiveTo: string`	`String effectiveTo`	ISO date.
`contractId: number`	`Long contractId`	FK pointer to the parent contract.

3.4 · Frontend wiring

CLAUSES.SERVICE.TS

```
private base = `${environment.apiBase}/clauses`;

list()   → GET  /clauses/get   .pipe(map(r => r.data))   // APIResponse<Clause[]>
create() → POST /clauses/post  .pipe(map(r => r.data))   // APIResponse<Clause>
```

CLAUSES.COMPONENT.TS

- Reuses the same Material dialog pattern as contracts. The form has fields for `clauseType`, `detailsJSON` (a textarea), `effectiveFrom` / `effectiveTo` (date pickers), and `contractId` (number).
- No delete or update endpoints exist on the backend yet, so the UI offers only list + create.

3.5 · Route & sidebar visibility

- Route: `/clauses` guarded by `roleGuard(['ADMIN','RIGHTS_MANAGER','SCHEDULER','COMPLIANCE_OFFICER'])`.
- Sidebar item appears under the **Rights** section, hidden for roles not in that list.

4 · Partner Service

Partners are downstream OTT/distribution endpoints (Acme TV, BigStream, etc.) that StreamCast pushes manifests to. A partner record is mostly contact + endpoint metadata.

4.1 · Service facts

Eureka name	`Partner-Service`
Port (direct)	`8087`
Database	`partner_database` (MySQL)
Base URL via gateway	`http://localhost:8082/partners`

4.2 · Endpoints

Verb	Path	Roles allowed	Description
POST	`/partners`	ADMIN DISTRIBUTION_OPERATOR	Create a partner.
GET	`/partners`	ADMIN DISTRIBUTION_OPERATOR PARTNER_ADMIN RIGHTS_MANAGER	List all partners.

4.3 · Field-by-field mapping

UI interface (`Partner`)	Backend DTO (`PartnerRequestDTO`)	Notes
`partnerId?: number`	(returned)	PK, auto-assigned.
`name: string`	`String name`	Partner display name.
`contactInfo: string`	`String contactInfo`	Email / phone / ops contact.
`endpointDetailsNote: string`	`String endpointDetailsNote`	Free-form — SFTP host, S3 bucket, API base, etc.
`status: 'ACTIVE' | 'INACTIVE'`	`String status`	Drives whether new manifests can be queued against this partner.
`titleId: number`	`long titleId`	The partner-title association the page was created for.

4.4 · Frontend wiring

PARTNERS.SERVICE.TS

```
private base = `${environment.apiBase}/partners`;

list()   → GET  /partners      .pipe(map(r => r.data))   // APIResponse<Partner[]>
create() → POST /partners      .pipe(map(r => r.data))   // APIResponse<Partner>
```

PARTNERS.COMPONENT.TS

- List page lives at `/partners` under `roleGuard(['ADMIN','DISTRIBUTION_OPERATOR','PARTNER_ADMIN','RIGHTS_MANAGER'])`.
- Renders a card grid: name + status chip + endpoint note + contact info.
- A "New partner" dialog (visible only to ADMIN / DISTRIBUTION_OPERATOR) posts to `create()` and re-runs `list()`.

4.5 · How the partner data flows into Distribution

The Partner page is read-only for the distribution flow. When a user creates a *manifest* on the Manifests page they enter a `partnerId` — that value comes from the partners list. There is no foreign-key constraint across databases (each service has its own DB); the link is logical only.

5 · Distribution Delivery Service

The richest of the four. Manages **manifests** (a package of assets + destination), **attempts** (each delivery try), and **receipts** (acknowledgements from the partner). Unlike the other three services, this one does *not* wrap responses in `APIResponse` — it returns raw entities.

5.1 · Service facts

Eureka name	`Distribution-Delivery`
Port (direct)	`8083`
Database	`distribution_delivery_database` (MySQL)
Base URL via gateway	`http://localhost:8082/manifests`
Response shape	Raw `Manifest` / `String` — **no** envelope.

5.2 · Endpoints — with role gates

Verb	Path	Roles allowed	Description
POST	`/manifests`	ADMIN DISTRIBUTION_OPERATOR	Create a manifest.
GET	`/manifests`	ADMIN DISTRIBUTION_OPERATOR PARTNER_ADMIN	List all manifests.
GET	`/manifests/{id}`	same as above	Fetch one manifest.
PUT	`/manifests/{id}`	ADMIN DISTRIBUTION_OPERATOR	Update fields (often status).
POST	`/manifests/{id}/attempts`	ADMIN DISTRIBUTION_OPERATOR	Record a delivery attempt.
POST	`/manifests/{id}/receipt`	ADMIN DISTRIBUTION_OPERATOR PARTNER_ADMIN	Record a partner acknowledgement.
GET	`/manifests/partner/{partnerId}`	same as list	All manifests for one partner.

5.3 · Field-by-field mapping — manifest body

UI form control	UI interface (`Manifest`)	Backend DTO (`ManifestDTO`)	Notes
Title ID	`titleId: string`	`String titleId`	Stored as string to allow non-numeric identifiers.
Asset IDs (JSON array)	`assetIdsJSON: string`	`String assetIdsJSON`	Free-form, e.g. `[1,2,3]` .
Destination	`destination: string`	`String destination`	e.g. `s3://bucket/path` .
Created by	`createdBy: string`	`String createdBy`	Usually the logged-in user's email.
Status (dropdown)	`status: string`	`String status`	QUEUED / IN_PROGRESS / DELIVERED / FAILED.
Partner ID	`partnerId: number`	`Long partnerId`	References a row in Partner-Service.

5.4 · The two side payloads — attempts & receipts

ATTEMPTDTO (POST /MANIFESTS/{ID}/ATTEMPTS)

```
{
  "result"  : "SUCCESS" | "FAILED",
  "details" : "Uploaded 4 files, 5.4 GB"
}
```

RECEIPTDTO (POST /MANIFESTS/{ID}/RECEIPT)

```
{
  "receivedBy" : "partner.ops@acme.com",
  "receiptURI" : "s3://receipts/m1.pdf"
}
```

5.5 · Frontend wiring

MANIFESTS.SERVICE.TS

```
private base = `${environment.apiBase}/manifests`;

list()        → GET  /manifests              // returns Manifest[]
get(id)       → GET  /manifests/{id}         // returns Manifest
create(m)     → POST /manifests              // returns Manifest
recordAttempt → POST /manifests/{id}/attempts// body: { status }, response: text
```

> **Body mismatch worth knowing:** the UI's `recordAttempt(id, status)` currently posts `{ status }` , but the backend's `AttemptDTO` expects `{ result, details }` . The field will arrive as `null` on the server. If you start using attempts from the UI, change the UI body to `{ result: status, details: '...' }` — or relax the DTO. There is no UI call for `/receipt` yet.

5.6 · Frontend component

- **ManifestsComponent** — route `/manifests` , guarded by `roleGuard(['ADMIN','DISTRIBUTION_OPERATOR','PARTNER_ADMIN'])` .
- Table columns: ID, Title, Partner, Destination, Status (coloured chip), Created at, Actions.
- **ManifestFormDialog** — reactive form with Title ID, Partner ID, Asset IDs (JSON), Destination, Created by, Status. Posts via `manifestsService.create()` .

5.7 · End-to-end — recording a delivery attempt

```
1. Operator clicks "Mark delivered" on a row in /manifests
2. UI calls manifestsService.recordAttempt(id, 'SUCCESS')
3. authInterceptor attaches the JWT
4. Gateway accepts, forwards to lb://Distribution-Delivery
5. Spring Security checks ROLE_ADMIN or ROLE_DISTRIBUTION_OPERATOR
6. DistributionController.attempt(id, dto) -> service.addAttempt(id, dto)
7. Returns plain text "Attempt recorded" (responseType: 'text' on the UI)
8. UI snackbars success, re-fetches the manifest row
```

6 · Cross-cutting concerns

6.1 · Error handling

- **UI side:** every `.subscribe()` has an `error: err => ...` handler that pops a `MatSnackBar` with `err?.error?.message` if present, falling back to a generic message. 401s are special-cased by the interceptor to log out and route to `/login`.
- **Gateway:** rejects bad/expired JWT with 401 (empty body) before reaching the service.
- **Service:** each has a `GlobalExceptionHandler` that turns `BadRequestException` → 400, `ResourceNotFoundException` → 404, and validation failures → 400 with a per-field map.

6.2 · CORS

Configured exactly once at the gateway (`CorsConfig.java`):

- Allowed origin: `http://localhost:4200`
- Methods: GET, POST, PUT, DELETE, PATCH, OPTIONS
- Exposed header: `Authorization`
- De-dupe filter strips duplicate `Access-Control-Allow-Origin` headers in case downstream services also set them.

6.3 · JWT secret & expiry

All four services and the gateway share the same secret string:

```
jwt.secret=Streamcast_Secure_Key_2026_Project_Very_Strong_Key_12345
jwt.expiration.ms=3600000   // 1 hour
```

The Angular `AuthService` decodes the JWT in `jwt.utils.ts` and stores `{ email, role, token, expiresAt }` in `localStorage` under the key `sc.auth`. A computed signal `isAuthenticated()` watches the expiry.

6.4 · Common payload shapes at a glance

Service	Response envelope?	Role enforcement	UI unwrap
Contract	Yes — `APIResponse<T>`	controller `@PreAuthorize`	`map(r => r.data)`
Clause	Yes — `APIResponse<T>`	controller `@PreAuthorize`	`map(r => r.data)`
Partner	Yes — `APIResponse<T>`	controller `@PreAuthorize`	`map(r => r.data)`
Distribution	**No** — raw entity / text	controller `@PreAuthorize`	none — direct cast

6.5 · Putting it all together — a request you can trace by hand

```
User Action               Network                    Backend
-----------               -------                    -------
Click "Save" on    POST /contracts               ContractController
ContractFormDialog  Host: localhost:8082              .createContract(dto)
                    Authorization: Bearer eyJ...          |
                    Content-Type: application/json        v
                    Body: { titleId: 12, ... }      @PreAuthorize gate
                          |                               |
                          v                               v
                    [JwtAuthFilter]                 ContractServiceImpl
                    valid? yes                      .saveContract(entity)
                    role = RIGHTS_MANAGER                 |
                    -> sets X-User-Role                   v
                                                    ContractRepository
                                                    .save(entity)
                                                          |
                                                          v
                                                    INSERT INTO contract...
                                                          |
                                                          v
                                                    APIResponse(
                                                      message="Contract created",
                                                      data=ContractResponseDTO{...},
                                                      success=true)
                          |
                          v
                    ContractsService.list()               |
                    refreshes the page                    v
                    snackbar "Contract created"     <-- 200 OK + body
```